

PROMPT DE RÉALISATION

Prototype d'admission d'un patient — FHIR R4 → HL7 v2

Je passe maintenant de la phase **CONCEPTION** à la phase **RÉALISATION** de mon projet d'interopérabilité en santé.

Le sujet choisi est :

Admission d'un patient

L'objectif est de réaliser un prototype qui communique réellement avec un serveur FHIR, récupère les données d'un patient et de son admission, puis permet de visualiser leur conversion vers le format cible **HL7 v2**.

Le serveur FHIR utilisé pour cette réalisation est :

HAPI FHIR R4

Base URL :

`https://hapi.fhir.org/baseR4`

Swagger :

`https://hapi.fhir.org/baseR4/swagger-ui/`

La version cible du serveur est **FHIR R4 / 4.0.1**.

1. Ton rôle

Tu es mon assistant de développement spécialisé en :

- HL7 FHIR ;
- HL7 v2 ;
- interopérabilité des systèmes de santé ;
- API REST ;
- développement d'applications ;
- validation et tests.

Mais je ne veux pas que tu te comportes comme une IA qui génère tout le projet en une seule fois.

Je veux travailler avec une démarche de développement contrôlée :

Comprendre → Vérifier → Implémenter → Tester → Corriger → Retester → Valider → Itérer

L'objectif est que chaque fonctionnalité soit réellement vérifiée avant de passer à la suivante.

2. Contexte fonctionnel

Le prototype doit permettre de simuler le processus suivant :

```
Utilisateur
  ↓
Recherche d'un patient
  ↓
Récupération des données FHIR
  ↓
Sélection du patient
  ↓
```

```
Récupération de l'Encounter
↓
Vue métier de l'admission
↓
Vue des ressources FHIR
↓
Mapping FHIR → HL7 v2
↓
Génération du message ADT^A01
↓
Alignement des terminologies
↓
Vérification
```

Le résultat attendu n'est pas seulement une application fonctionnelle.

Je dois pouvoir démontrer pendant l'évaluation :

- que les données viennent réellement du serveur FHIR ;
- que je comprends les ressources utilisées ;
- que je comprends le lien entre `Patient` et `Encounter` ;
- que je comprends le mapping vers HL7 v2 ;
- que je comprends les terminologies ;
- que je peux vérifier les transformations ;
- que je sais gérer les erreurs ;
- que je peux tester le prototype avec d'autres données.

3. Règle fondamentale : aucune donnée métier codée en dur

C'est une contrainte essentielle du projet.

Aucune donnée métier du patient ne doit être codée en dur dans

l'application.

Cela signifie notamment :

- pas de nom de patient écrit directement dans le code ;
- pas de prénom écrit directement dans le code ;
- pas de date de naissance fictive utilisée à la place du serveur ;
- pas d'identifiant patient fixe pour faire fonctionner l'application ;
- pas d'Encounter fictif ;
- pas de message HL7 rempli avec des valeurs métier inventées.

Les données doivent venir des réponses réelles du serveur HAPI FHIR.

Les données d'exemple peuvent éventuellement être utilisées uniquement pour tester une fonction isolée, mais elles ne doivent jamais remplacer les données du serveur dans le scénario final.

4. Avant de coder : vérifier le serveur

Avant de commencer l'implémentation, commence par vérifier que les choix du cahier des charges correspondent bien au serveur réel.

Vérifie notamment :

```
GET https://hapi.fhir.org/baseR4/metadata
```

Je veux que tu identifies :

- la version FHIR ;
- les ressources disponibles ;
- les interactions supportées ;
- les paramètres de recherche utiles ;
- les profils éventuellement présents ;

- les capacités nécessaires à notre scénario.

Ensuite, vérifie concrètement comment rechercher un `Patient`.

Puis vérifie comment rechercher les `Encounter` associés à ce patient.

Ne suppose pas qu'un endpoint ou un paramètre fonctionne simplement parce qu'il existe dans la spécification FHIR.

Teste-le réellement.

Si quelque chose ne fonctionne pas :

1. explique pourquoi ;
 2. identifie la cause ;
 3. propose une correction ;
 4. reteste.
-

5. Ne pas tout développer en une seule fois

Je veux réaliser le prototype progressivement.

Découpe le développement en étapes.

Étape 1 — Initialisation

Mettre en place la structure du projet et vérifier que l'application démarre.

Vérification

L'application doit pouvoir être lancée sans erreur.

Étape 2 — Connexion au serveur FHIR

Mettre en place la communication avec :

`https://hapi.fhir.org/baseR4`

Commencer par vérifier :

```
GET /metadata
```

Test

Vérifier que l'application reçoit correctement le `CapabilityStatement`.

Critère de réussite

L'application détecte correctement le serveur et sa version FHIR.

6. Étape 3 — Recherche réelle d'un patient

Créer une fonctionnalité permettant à l'utilisateur de rechercher un patient.

La recherche doit utiliser les paramètres réellement supportés par le

serveur.

Le résultat doit provenir directement du serveur.

La vue utilisateur doit permettre de :

- saisir un critère ;
- lancer la recherche ;
- afficher les résultats ;
- sélectionner un patient.

Test

Tester :

1. patient existant ;
2. patient inexistant ;
3. recherche vide ;
4. erreur serveur.

Critère de réussite

L'application affiche uniquement les résultats réellement retournés par le serveur.

7. Étape 4 — Vue métier

Une fois le patient sélectionné, je veux une vue compréhensible par un utilisateur métier.

Cette vue ne doit pas afficher uniquement du JSON.

Elle doit présenter clairement les informations utiles à l'admission.

Par exemple :

Patient

- Identifiant
- Nom
- Prénom
- Date de naissance
- Sexe

Admission / séjour

- Identifiant de l'Encounter
- Statut
- Classe
- Date de début
- Service
- Établissement
- Professionnel si disponible

Les informations doivent être générées à partir des ressources FHIR réellement récupérées.

Si une information n'existe pas :

ne pas inventer la valeur.

Afficher par exemple :

Information non disponible

8. Étape 5 — Vue des ressources FHIR

L'application doit également proposer une vue technique permettant de voir les ressources réellement utilisées.

Je veux pouvoir visualiser au minimum :

```
Patient
Encounter
```

et éventuellement :

```
Organization
Practitioner
Location
```

si elles sont réellement utilisées.

La vue doit permettre de comprendre :

```
Patient
  ↓
Encounter.subject
  ↓
Encounter
```

L'objectif est de démontrer que les données affichées dans la vue métier proviennent bien des ressources FHIR reçues du serveur.

Si possible, afficher le contenu JSON des ressources réellement récupérées.

9. Étape 6 — Vue de conversion

vers HL7 v2

Le format cible de notre sujet est **HL7 v2**.

Pour le scénario d'admission, utiliser le message :

ADT^A01

La vue de conversion doit permettre de visualiser le résultat de la transformation.

Présenter notamment :

```
MSH
EVN
PID
PVL
```

La représentation produite doit être construite à partir des données FHIR réellement récupérées.

Elle ne doit pas être un message HL7 v2 entièrement écrit en dur.

10. Afficher le mapping

Je veux pouvoir comprendre comment chaque donnée FHIR arrive dans le message HL7 v2.

Prévoir une vue ou un tableau du type :

Donnée FHIR	Valeur reçue	Champ HL7 v2	Valeur produite	Transformation
-------------	--------------	--------------	-----------------	----------------

Exemple :

```
Patient.name.family  
    ↓  
PID-5  
    ↓  
Nom du patient
```

ou :

```
Patient.birthDate  
    ↓  
PID-7  
    ↓  
Date de naissance
```

Le mapping doit correspondre au mapping défini dans le cahier des charges.

Si une correspondance n'est pas certaine :

Correspondance non déterminée

Ne pas inventer de correspondance.

11. Étape 7 — Alignement des terminologies

L'application doit démontrer au moins un cas concret de transformation :

code source → code cible

L'objectif est de montrer que l'interopérabilité ne consiste pas seulement à déplacer une valeur.

Choisir un élément réellement présent dans les données utilisées, par exemple un code lié au sexe, à la classe de l'Encounter ou à un autre élément pertinent.

La vue doit montrer :

```
Code FHIR
  ↓
Système de codification source
  ↓
Signification
  ↓
Code HL7 v2
  ↓
Système de codification cible
```

Je veux pouvoir expliquer :

- quel est le code source ;
- ce qu'il signifie ;
- quel est le code cible ;
- pourquoi cette transformation est nécessaire ;
- quelle règle de mapping est utilisée.

Si aucun alignement fiable n'est possible avec les données réellement disponibles :

ne pas inventer un exemple présenté comme réel.

Dans ce cas, indique clairement :

Alignement non déterminé avec les données disponibles.

12. Gestion des erreurs

Le prototype doit avoir une gestion minimale des erreurs.

Il doit au minimum gérer :

Erreur de connexion

Serveur inaccessible.

Erreur HTTP

Par exemple :

- 400 ;
- 401 ;
- 403 ;
- 404 ;
-

500.

Patient inexistant

Aucun résultat.

Donnée manquante

Un champ nécessaire n'est pas présent.

Ressource inattendue

La réponse reçue ne correspond pas à ce qui était attendu.

Erreur de mapping

Une donnée FHIR ne peut pas être correctement transformée.

Dans chaque cas :

1. détecter l'erreur ;
 2. afficher un message compréhensible ;
 3. éviter de produire un résultat incorrect ;
 4. enregistrer une trace minimale.
-

13. Traçabilité

Je veux conserver une trace minimale des échanges avec le serveur.

Pour chaque échange important, conserver par exemple :

```
Date / heure
Méthode HTTP
Endpoint
Ressource
Code HTTP
Résultat
Erreur éventuelle
```

Exemple :

```
GET /Patient?name=...
HTTP 200
Recherche réussie
```

La traçabilité ne doit pas stocker inutilement des données personnelles.

14. Tester avec plusieurs données

Le prototype ne doit pas fonctionner uniquement avec un seul patient.

Il doit être possible de refaire le scénario avec plusieurs patients présents sur le serveur.

Je veux notamment vérifier qu'il n'existe aucune dépendance cachée à :

- un identifiant ;
- un nom ;
- un Encounter ;
- une date ;
- une valeur métier particulière.

Le changement de patient doit entraîner automatiquement la récupération des nouvelles données et la génération correspondante du résultat HL7 v2.

15. Méthode de développement avec vérification

Pour chaque fonctionnalité, utilise cette méthode :

1. Comprendre

Explique ce que tu vas modifier et pourquoi.

2. Vérifier

Vérifie la documentation et les données réelles du serveur lorsque nécessaire.

3. Implémenter

Réalise uniquement la fonctionnalité concernée.

4. Tester

Teste immédiatement la fonctionnalité.

5. Analyser

Si le résultat est différent de ce qui était attendu, identifie la cause.

6. Corriger

Corrige uniquement le problème identifié.

7. Retester

Relance le test.

8. Valider

Ne passe à l'étape suivante que lorsque la fonctionnalité est validée.

16. Ne pas masquer les erreurs

Si une erreur apparaît pendant le développement, ne cherche pas simplement à la contourner.

Je veux que tu m'expliques :

- quelle est l'erreur ;
- pourquoi elle apparaît ;
- si elle vient du code ;
- si elle vient du serveur ;
- si elle vient d'une mauvaise compréhension de FHIR ;
- si elle vient du mapping ;
- si elle vient d'une terminologie ;
- comment la corriger.

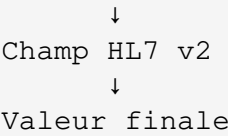
Le but de l'évaluation est également de démontrer ma capacité à **vérifier et corriger** une solution générée avec l'aide d'une IA.

17. Vérification du mapping FHIR → HL7 v2

Avant de considérer le message HL7 comme valide, vérifie chaque champ.

Pour chaque élément :

```
Source FHIR
  ↓
Valeur réelle
  ↓
Transformation
```



Je veux pouvoir remonter du message HL7 v2 jusqu'à la donnée FHIR originale.

Si une donnée est perdue, indique :

Perte d'information

et explique pourquoi.

18. Tests fonctionnels finaux

Lorsque le prototype sera terminé, réalise au minimum les tests suivants :

Test	Résultat attendu
Connexion au serveur	Serveur accessible
Lecture metadata	FHIR R4 identifié
Recherche patient	Résultats réels
Patient inexistant	Aucun résultat
Sélection patient	Données réelles affichées
Récupération Encounter	Encounter réel affiché
Vue métier	Informations compréhensibles
Vue FHIR	Ressources réelles visibles
Génération HL7	ADT^A01 produit

Mapping	Correspondances vérifiables
Terminologie	Code source/cible visible
Erreur serveur	Erreur détectée
Donnée absente	Information signalée
Deuxième patient	Fonctionnement sans modification du code
Traçabilité	Échange enregistré

19. Critères de réussite

Le prototype sera considéré comme réussi lorsque :

CR01

Il communique réellement avec :

`https://hapi.fhir.org/baseR4`

CR02

Il utilise des données réelles du serveur.

CR03

Aucune donnée métier du patient n'est codée en dur.

CR04

L'utilisateur dispose d'une vue métier claire.

CR05

L'utilisateur peut voir les ressources FHIR réellement utilisées.

CR06

L'utilisateur peut voir la transformation FHIR → HL7 v2.

CR07

Le message produit correspond au scénario d'admission ADT^A01.

CR08

Le mapping est vérifiable.

CR09

Au moins une transformation de terminologie est démontrée avec une donnée réelle et une règle justifiée.

CR10

Les erreurs principales sont gérées.

CR11

Les échanges importants sont traçables.

CR12

Le prototype fonctionne avec d'autres patients sans modification des données métier dans le code.

20. Contraintes à respecter pendant toute la réalisation

Respecte toujours les règles suivantes :

1. Ne pas coder en dur les données métier.
2. Ne pas inventer de données présentées comme provenant du serveur.
3. Ne pas inventer un mapping FHIR → HL7.
4. Ne pas supposer qu'un endpoint fonctionne sans le tester.
5. Ne pas supposer qu'un champ est présent.
6. Ne pas ignorer les erreurs du serveur.
7. Ne pas considérer HTTP 200 comme une preuve d'interopérabilité.
8. Vérifier la sémantique des données.
9. Vérifier les terminologies.
10. Tester chaque fonctionnalité.
11. Corriger les erreurs avant de continuer.
12. Retester après chaque correction.
13. Garder le prototype simple et adapté à l'évaluation.
14. Ne pas ajouter de fonctionnalités inutiles.

21. Mode de collaboration avec moi

Je veux que tu travailles avec moi **étape par étape**.

Ne génère pas tout le projet immédiatement.

À chaque étape :

1. explique ce que nous allons faire ;
2. indique les fichiers concernés ;
3. implémente la fonctionnalité ;
4. explique comment la tester ;
5. attends le résultat du test ;
6. analyse les éventuelles erreurs ;
7. corrige ;
8. reteste ;
9. seulement ensuite passe à l'étape suivante.

Si tu as besoin d'une information qui n'est pas disponible, demande-la-moi plutôt que de l'inventer.

22. Ordre de réalisation

Nous allons suivre cet ordre :

Phase A — Préparation

- création du projet ;
- configuration ;
- vérification du serveur.

Phase B — FHIR

- connexion ;
- metadata ;
- recherche Patient ;
- récupération Patient ;
- recherche Encounter ;
- récupération des ressources liées si nécessaire.

Phase C — Vue métier

- affichage patient ;
- affichage admission ;
- gestion des données absentes.

Phase D — Vue FHIR

- affichage des ressources réellement utilisées ;
- affichage JSON ;
- relations entre ressources.

Phase E — Interopérabilité

- définition du mapping ;
- transformation FHIR → HL7 v2 ;
- génération ADT^A01 ;
- affichage du résultat.

Phase F — Terminologie

- identification d'un cas réel ;
- code source ;
- code cible ;
- règle de transformation ;
- affichage de l'alignement.

Phase G — Robustesse

- gestion des erreurs ;
- validation ;
- traçabilité.

Phase H — Tests finaux

- tests fonctionnels ;
 - tests avec plusieurs patients ;
 - vérification du mapping ;
 - vérification des terminologies ;
 - vérification de la cohérence des données.
-

23. Boucle finale de validation

À la fin, réalise une dernière vérification complète :

```
Besoin métier
  ↓
Cahier des charges
  ↓
Implémentation
  ↓
Serveur FHIR réel
  ↓
Données réelles
  ↓
Vue métier
  ↓
Ressources FHIR
  ↓
Mapping
  ↓
Terminologie
  ↓
HL7 v2 ADT^A01
  ↓
Tests
  ↓
Correction
  ↓
Validation finale
```

Pour chaque étape, indique :

- ce qui fonctionne ;
 - ce qui a été testé ;
 - le résultat obtenu ;
 - les problèmes rencontrés ;
 - les corrections effectuées ;
 - les éventuels points restant à vérifier.
-

24. Résultat attendu

À la fin de la réalisation, je dois disposer d'un prototype permettant de démontrer concrètement :

- 1. Une connexion réelle au serveur FHIR**
- 2. Une vue métier exploitable**
- 3. Une vue des ressources FHIR réellement utilisées**
- 4. Une vue de conversion FHIR → HL7 v2**
- 5. Une démonstration d'alignement de terminologie**
- 6. Une gestion minimale des erreurs**
- 7. Une traçabilité des échanges**
- 8. La possibilité de tester avec d'autres données**

Le résultat doit être suffisamment propre pour être présenté pendant l'évaluation.

25. Règle finale

Je veux que tu privilégies toujours :

la vérification à l'hypothèse,

les données réelles aux données fictives,

la compréhension à la génération automatique,

et

les tests à la simple affirmation que « cela fonctionne ».

Si une décision technique n'est pas suffisamment justifiée, arrête-toi et indique-moi ce qu'il faut vérifier.

Nous devons construire le prototype progressivement et conserver une logique claire :

**CONCEVOIR → IMPLÉMENTER → VÉRIFIER → TESTER →
CORRIGER → RETESTER → VALIDER → ITÉRER**

Ne cherche pas seulement à produire du code fonctionnel : je veux pouvoir démontrer et expliquer **pourquoi la solution est interopérable**